Axial take-home assignment
The Quote & State Engine

The reality: order-to-cash chaos

Currently, Izzy, a sales rep at one of our customers (an industrial materials distributor), experiences a fragmented WhatsApp request. She manually queries their ERP to check inventory, then generates an "Oferta de Venta" (sales quote). The conversation doesn't stop there: the customer revises quantities, a colleague of theirs asks about the same quote from a different chat, and new requests arrive mixed in with old ones. Izzy keeps all of this straight in her head. It's an operational nightmare of context-switching and manual data entry.

The challenge?

Timebox: 48 hours total (*but* please don't spend more than ~6 hours of active work, remember we want to see your 80/20 execution).

Build a headless pipeline i.e. clean API that does two things:

1. Turns a colloquial WhatsApp conversation into a structured draft quote.
2. **Keeps that quote correct as messages keep arriving** — revisions, questions from a second chat, and new requests — without mixing them up.

Phase 2 is where the engineering is so please plan to spend most of your time there. Phase 1's extraction only needs to be good enough to feed it. Pricing is out of scope: a quote carries SKUs and quantities; the ERP prices it later.

The catalog (mock DB):

SKU	**Description**
ALFBTC-013-320100	Lona Frontlit BTE 13 3.20x100
ALMCLC-0909-320100	Lona Mesh CLN 9x9 3.20x100
AVMICS-1A-137050	Vinil Adhesivo Microperforado 1.37x50
ARACBL-03-122244	Acrilico Blanco 3mm 1.22x2.44

Phase 1: unstructured request → structured quote

The input — a WhatsApp conversation:

```JSON
{
  "customer_id": "C-10029",
  "conversation_id": "wa-thread-411",
  "customer_name": "Alice - ACME INDUSTRIAL",
  "conversation_history": [
    {"direction": "inbound",  "timestamp": "2026-02-10T09:43:00Z",
     "text": "Hola buenos dias, me ayudas a cotizar 20 rollos lona 3.20 x 100 m.,
5 rollos de mesh de 3.20 x 100 m. y si cuentas con microperforado de 1.37 y 1.50
m."},
    {"direction": "outbound", "timestamp": "2026-02-10T09:45:00Z",
     "text": "Hola buen día. Si. Cuantas piezas de microperforado"},
    {"direction": "outbound", "timestamp": "2026-02-10T09:45:30Z",
     "text": "De momento sería de 1.37x50"},
    {"direction": "inbound",  "timestamp": "2026-02-10T09:46:00Z",
     "text": "Necesitamos 5 rollos"}
  ]
}
```

Note that the details of one line item arrive scattered across the conversation — the product in one message, the size in another (stated by the *rep*, answering availability), the quantity in a third. Your pipeline must read both directions to assemble it. One more thing to notice: the customer also asked about a 1.50 m width. The rep answered that in-chat — only 50 is available — so it needs nothing from your pipeline.

The expected output:

```JSON
{
  "document_type": "Oferta_de_Venta",
  "quote_id": "Q-001",
  "status": "draft",
  "conversations": ["wa-thread-411"],
  "line_items": [
    {"sku": "ALFBTC-013-320100", "qty": 20},
    {"sku": "ALMCLC-0909-320100", "qty": 5},
    {"sku": "AVMICS-1A-137050", "qty": 5}
  ]
}
```

(Your ID format can differ — we use Q-001 and Q-002 below to refer to the quotes.)

Phase 2: the conversation keeps going

Here is the part Izzy actually struggles with. After Q-001 exists, four more events arrive — **as four separate calls to your pipeline, in this order**. Your system's stored state must be correct after each one. (State must live somewhere that survives between calls — and a process restart.) Every event is one message, shaped like this:

```
JSON
{"message_id": "wamid.411.0231", "conversation_id": "wa-thread-411",
"customer_id": "C-10029",
 "direction": "inbound", "timestamp": "2026-02-11T10:14:00Z",
 "text": "Oye de las lonas mejor que sean 15, no 20"}
```

Event 1 — the message above, in Alice's original chat.

Expected: the 20-roll lona line in Q-001 becomes qty 15. Nothing else changes.

Event 2 — a **different chat** (`wa-thread-662`), new contact at the same customer:

> "Buen dia, soy Mario de compras de ACME. Alice me pasó con ustedes, me confirmas qué lleva la cotización de las lonas?"

Expected: no line changes. Your system works out that this chat is talking about Q-001 and remembers it — from now on, both conversations are linked to that quote. (How it works that out is yours to design; what matters is that your stored state knows it afterward.)

Event 3 — back in Alice's chat:

> "y aparte cotizame 2 acrilicos blancos de 3mm porfa"

Expected: a **new** quote, Q-002 (she said "aparte" — separate), with 2× ARACBL-03-122244. One chat now feeds two quotes.

Event 4 — Mario's chat:

> "perfecto. y de los acrilicos que cotizó Alice, mejor que sean 4"

Expected: Q-002's acrilico line becomes qty 4 — a revision to one quote, arriving from the *other* conversation. The message says which quote it means ("los acrilicos"); the chat it arrived in does not decide.

Expected final state (returned by something like `GET /quotes?customer=C-10029`):

```json
{
  "quotes": [
    {"quote_id": "Q-001", "status": "draft",
     "conversations": ["wa-thread-411", "wa-thread-662"],
     "line_items": [
       {"sku": "ALFBTC-013-320100", "qty": 15},
       {"sku": "ALMCLC-0909-320100", "qty": 5},
       {"sku": "AVMICS-1A-137050", "qty": 5}
     ]},
    {"quote_id": "Q-002", "status": "draft",
     "conversations": ["wa-thread-411", "wa-thread-662"],
     "line_items": [
       {"sku": "ARACBL-03-122244", "qty": 4}
     ]}
  ]
}
```

Quotes should keep enough history that a human can see what changed and which message caused it.

What we are evaluating

1. **Architecture & pragmatism.** How do you balance deterministic logic with probabilistic LLMs? We want to see how you prevent the LLM from burning tokens on simple tasks — or hallucinating SKUs. A SKU that isn't in the catalog should be impossible in your output, by construction.

2. **The data model.** Look at what the four events do to your storage: two different chats end up feeding Q-001, and Alice's single chat ends up feeding two quotes. The two obvious shortcuts both break — if a conversation carries a `quote_id`, event 3 breaks it; if a quote carries a `conversation_id`, event 2 breaks it. We want to see how you store conversations, quotes, and the connection between them so that none of the four events needs a workaround. Show the model in your README (a simple diagram or table sketch is perfect), and walk us through the moment a new message arrives: what does your system actually look at to decide which quote it belongs to? Event 4 is the test — the answer can't be "whichever chat it came from."

3. **Knowing when not to guess.** Suppose a fifth message arrived: *"mejor cancelalo"* — with two open quotes, it's genuinely unclear which one. In your README, a short

paragraph: what does your system do with a message it can't confidently assign? (Hint: pushing a guess into the ERP is the wrong answer.)

Extra credit (optional)

Only if you're comfortably inside the timebox — we'd much rather see a clean core than a sprawling extra. Two small ones, in order of value:

1. **Duplicate delivery.** WhatsApp webhooks sometimes deliver the same event twice. Re-send event 4 to your pipeline, byte-identical, same `message_id`: your state should not change again. (Careful: two *different* messages with the same text are a real repeat request — don't dedupe by text.)
2. **Make "cancelalo" real.** Instead of only describing the behavior from point 3 in your README, implement it: the message lands somewhere a human can see it, with the quotes it might refer to, and nothing changes until a person decides.

Submission guidelines

Send over a GitHub repository. No UI — we read the README and run the pipeline. Your README should contain, briefly:

1. Your data model — the sketch from point 2 above, and the walkthrough of how a new message finds its quote.
2. The "mejor cancelalo" paragraph from point 3.
3. Your architectural trade-offs, and how you would scale this if Acme started processing 1,000 conversations a day (that's less than one per minute — think about what actually breaks first).
4. One honest paragraph on how you used AI tools — we expect you did.

Brief beats complete: a page or two of README is plenty. If you did the extra credit, add one line each telling us how to see it.